

HDT5 — Orquestación multiagente: Parachute S.A.

Análisis comparativo de arquitecturas centralizada, jerárquica y descentralizada

Parachute S.A. — Proyecto HDT5

2026-09-17

1. Contexto y metodología

Se implementó el mismo problema de negocio (responder FAQs del evento y gestionar citas de salto validando el clima con Open-Meteo) en tres arquitecturas de agentes distintas —centralizada, jerárquica y descentralizada—, todas construidas sobre el OpenAI Agents SDK y un mismo proveedor de modelo (Groq, `openai/gpt-oss-20b`), reutilizando exactamente el mismo núcleo de dominio, integraciones y tools (`src/domain`, `src/services`, `src/integrations`, `src/tools`).

Las conclusiones de este documento no son solo teóricas: se basan en (a) una suite de 73 tests automatizados deterministas (dominio, integraciones, flujo/tools compartido y cableado estructural de las tres arquitecturas) y (b) una corrida real end-to-end contra Groq y Open-Meteo (`docs/smoke-test-output.txt`) con el mismo guion de tres turnos para las tres arquitecturas: una pregunta de FAQ, una solicitud de reserva para una fecha con clima real prohibido, y una solicitud para una fecha fuera del horizonte de pronóstico.

2. Pregunta 1 — ¿Qué arquitectura resuelve mejor este problema? ¿Por qué?

2.1 Comparación

Criterio	Centralizada	Jerárquica	Descentralizada
Complejidad de implementación	Baja	Media	Alta
Control del workflow global	Alto (un solo agente decide todo)	Alto (dos niveles, pero explícito)	Bajo (emergente, depende de qué agente tenga el control en cada turno)
Trazabilidad de una conversación	Alta (un log de agente por turno)	Alta (dos niveles de log)	Media/baja (hay que reconstruir la cadena de handoffs)
Riesgo de rutas inesperadas	Bajo	Bajo	Medio-alto

Criterio	Centralizada	Jerárquica	Descentralizada
Resolución en un solo turno de peticiones combinadas (FAQ + reserva)	Sí, de forma nativa (un supervisor combina especialistas via <code>as_tool()</code> en la misma respuesta)	Sí, vía Booking/Manager	No garantizado: en la corrida real, el FAQ Agent respondió la pregunta de conocimiento y recién en el turno siguiente transfirió el control al Weather Agent, que en su primera respuesta solo anunció la transferencia sin evaluar aún la fecha
Extensibilidad (agregar un nuevo dominio, p.ej. pagos)	Requiere tocar al supervisor único	Se agrega un manager nuevo sin tocar los existentes	Se agrega un agente nuevo y sus handoffs, pero hay que revisar todos los puntos de entrada/salida ciclicos

Criterio	Centralizada	Jerárquica	Descentralizada
Facilidad de testing	Alta (un solo grafo de tools)	Alta (dos niveles, pero cada manager se prueba por separado)	Media (hay que fijar el schema de cada handoff; ver 2.3)
Número de agentes / coordinación necesaria	1 supervisor + 3 especialistas	1 root + 2 managers + 3 especialistas (6 agentes)	3 agentes, coordinación vía handoffs (sin agente permanente)

2.2 Evidencia observada

En la corrida real (`docs/smoke-test-output.txt`):

- **Centralizada** y **jerárquica** resolvieron los tres turnos de forma directa y en una sola respuesta cada uno: FAQ correcta, bloqueo correcto de la reserva cuando Open-Meteo devolvió condiciones reales PROHIBITED (24.6 mm de lluvia, 85% de nubes ese día), y rechazo correcto de la fecha fuera de horizonte (2027-04-05, más allá del último día válido 2026-10-02).
- **Descentralizada** resolvió los mismos tres escenarios correctamente en cuanto a *resultado final* (ninguna cita se creó de forma indebida), pero con más fricción conversacional: el turno de reserva requirió un handoff (FAQ Agent -> Weather Agent) cuya primera respuesta fue solo un anuncio de transferencia, no la evaluación del clima. Es decir, la misma tarea tomó más turnos/latencia en llegar al mismo resultado.
- En ningún caso, con ninguna arquitectura, se creó una cita para la fecha con clima PROHIBITED ni para la fecha fuera de rango: la garantía anti-bypass en `src/tools/calendar_tools.py` y `src/services/calendar_service.py` se sostuvo independientemente de la arquitectura, confirmando que la seguridad no depende de qué tan bien “razone” el LLM de turno.

2.3 Costo real de la descentralizada (evidencia adicional, no solo teoría)

Durante la implementación se encontró un problema real y no anticipado: Groq rechaza herramientas (`tools[].function.parameters`) cuyo JSON schema tiene `properties: {}` vacío junto con `required: []`, que es exactamente lo que el SDK genera para un `handoff()` sin `input_type`. Esto rompía los tres handoffs de la arquitectura descentralizada con un error 400 en el primer mensaje de cualquier conversación. La solución fue introducir un `HandoffReason` (un campo `reason: str` obligatorio) en cada handoff. Esto es evidencia concreta de que la arquitectura descentralizada, al depender de más mecanismos del SDK (handoffs) que centralizada/jerárquica (que solo usan `as_tool()`), tiene más superficie de compatibilidad con el proveedor de LLM y más piezas que pueden fallar de forma no obvia.

2.4 Conclusión

Para el alcance actual del problema (FAQs + una única cadena de decisión: validar fecha -> clima -> evaluación -> cita), **la arquitectura centralizada es la que mejor resuelve el problema hoy**: mismo resultado correcto que las otras dos, con la menor complejidad, el menor número de agentes, la mejor trazabilidad y sin los problemas de compatibilidad que mostró la descentralizada.

La **jerárquica** es la segunda mejor opción y se vuelve la más apropiada en cuanto el negocio explícitamente indique que van a **crecer los dominios** (el enunciado de Parachute S.A. lo advierte): agregar pagos, logística de transporte, o gestión de instructores como un tercer “Manager” no obliga a tocar el Root Manager ni a los managers existentes, algo que sí requeriría cambios en el supervisor único de la arquitectura centralizada.

La **descentralizada** demuestra desacoplamiento real (ningún agente conoce a todos los demás; el control se transfiere, no se delega y se retoma), pero para este problema concreto ese desacoplamiento no se traduce en una ventaja observable: se pagó en más turnos de conversación, más superficie de fallos de compatibilidad con el proveedor de LLM, y menor trazabilidad, sin que el negocio necesite hoy la autonomía que ese patrón ofrece (por ejemplo, agentes que puedan operar en procesos o equipos separados).

3. Pregunta 2 — ¿Es necesario un sistema multiagente en este caso?

3.1 “Puede resolverse con MAS” vs. “necesita MAS”

El problema funcional descrito por Parachute S.A. —responder FAQs y agendar una cita validando clima con reglas fijas— **no necesita** un sistema multiagente para resolverse correctamente. La evidencia de esta misma implementación lo confirma: toda la lógica que garantiza corrección y seguridad (`validate_forecast_date`, `assess_jump_conditions`, `InMemoryCalendarService.create_appointment`) es código determinista en `src/domain` y `src/services`, cubierto por 73 tests que **no dependen de ningún LLM ni de ninguna arquitectura de agentes**. Un único agente (o incluso un programa sin LLM, con un árbol de decisión) con acceso a esos mismos tools/servicios habría resuelto el mismo problema con el mismo nivel de corrección y seguridad.

Lo que el LLM aporta en las tres arquitecturas es exclusivamente la capa conversacional: entender lenguaje natural, decidir qué tool llamar y explicar el resultado en español. Ese valor existe con **un solo agente con varios tools**, no requiere múltiples agentes coordinándose.

3.2 Entonces, ¿por qué hacer el ejercicio multiagente?

El valor de las tres variantes multiagente en este proyecto es principalmente:

- **Separación de responsabilidades** explícita entre conocimiento (FAQ) y transacción (clima + reserva), útil si estos dominios crecen o se asignan a equipos distintos.
- **Experimentación de patrones de orquestación** (as_tool vs. handoffs) que sí importan cuando el número de especialistas y de reglas de negocio crece.
- **Preparación para crecimiento futuro**: Parachute S.A. advierte explícitamente que los requerimientos seguirán aumentando; la jerárquica en particular deja una estructura donde sumar dominios nuevos es barato.

3.3 El costo de multiagente, con evidencia de esta implementación

- **Más llamadas al modelo y más latencia**: en la corrida real, resolver la misma pregunta de FAQ tomó ~1.4s en centralizada (una sola llamada al supervisor con `as_tool()`) pero la reserva sobre una fecha con clima prohibido tomó ~14-24s en jerárquica (Root Manager -> Booking Manager -> Weather/Scheduling son varias llamadas anidadas al mismo modelo) frente a ~1s en centralizada para el mismo turno.
- **Más riesgo de incompatibilidad con el proveedor**: el bug real de handoffs vacíos con Groq (sección 2.3) solo aparece por usar el mecanismo de handoffs de la arquitectura descentralizada; una arquitectura de un solo agente nunca lo habría encontrado.
- **Más casos de prueba**: se necesitaron tests dedicados de *wiring* (`tests/agents/test_architecture_wiring.py`) solo para verificar que cada arquitectura está realmente cableada como se espera (supervisor con 3 tools, root manager con 2 tools sin acceso directo a especialistas, handoffs cíclicos correctos) —verificación que no existiría con un agente único.

- **Más superficie de observabilidad:** fue necesario instrumentar `architecture`, `agent`, `tool`, `handoff` por separado (`src/observability.py`) para poder comparar de forma objetiva las tres variantes; con un solo agente esa dimensión no existiría.
- **El LLM puede narrar mal un resultado correcto:** en la corrida real, el supervisor centralizado explicó el rechazo de la fecha fuera de rango diciendo “normalmente hasta 7-10 días” cuando el horizonte real y correcto es 15 días — una imprecisión conversacional que no afectó la decisión (el código de dominio, no el LLM, es quien decide), pero que ilustra el riesgo de dejar que un LLM explique reglas de negocio con sus propias palabras.

3.4 Conclusión

No, el problema tal como está descrito **no necesita** un sistema multiagente: se puede resolver, y de hecho se resuelve con mayor eficiencia, con un agente único apoyado en tools deterministas. El valor de las tres arquitecturas implementadas aquí es principalmente de **preparación organizacional y de aprendizaje de patrones**, dado el aviso explícito de Parachute S.A. de que los requerimientos seguirán creciendo. Si ese crecimiento se concreta (más dominios de negocio, más equipos, más especialistas), la arquitectura **jerárquica** es la que mejor absorbe ese crecimiento sin heredar los costos de compatibilidad y latencia observados en la descentralizada.